ECEN4223-5080 Spring 2026
Mobile Robotics
Project 2

Due date: Submit your report by 11:59 p.m. Tuesday, March 24th, 2026 to Canvas.

I. Objectives

1. Get familiar with the Gazebo simulation tool
2. Get familiar with how to configure and launch the Yahboom Rosmasterx3 simulation package
3. Learn the difference between the odometry position estimates and the ground truth position of the robot
4. Learn how to control robot motion using an open-loop approach and understand its limitations

II. Tasks | **Submission requirements are highlighted in bold and underlined text**

1. Build the robot simulation environment and platform.
   a. Install the Rosmasterx3 simulation package stack.
      For this task, please refer to **Appendix I**.
      <u>Submit a screenshot of the terminal showing the launch file is running</u>.

   b. Start the Gazebo simulation system, load the home environment and robot.
      For this task, please refer to **Appendix II**.
      <u>Submit a screenshot of the home environment and the Rosmasterx3 robot in the Gazebo window.</u>

2. Control the robot motion using a keyboard.
   a. Change the robot's initial position in Gazebo.
      - Understand the coordinate systems of Gazebo.
        For this step, please refer to **Appendix III**.
      - Change the robot's initial position to (-3, 2, 0.05) in the world coordinate system. You can find the setting file of the robot's position by editing the launch file located at:
        "~/ros2_ws/src/yahboom_rosmaster/yahboom_rosmaster_projects/launch/gz_home_like.launch.py".
        The robot's initial position is defined in the code section between lines 140 and 153 of this file.

```python
declare_x_cmd = DeclareLaunchArgument(
    name='x',
    default_value='0.5',
    description='x component of initial position, meters')

declare_y_cmd = DeclareLaunchArgument(
    name='y',
    default_value='0.0',
    description='y component of initial position, meters')

declare_z_cmd = DeclareLaunchArgument(
    name='z',
    default_value='0.05',
    description='z component of initial position, meters')
```

Submit a screenshot showing the robot's initial position has changed.

b. Based on the initial robot position set in the previous task, display and compare the robot's position using the odometry frame and the world (ground-truth) frame.

- Open a terminal and run the following command:
  $: ros2 topic echo /mecanum_drive_controller/odom --field pose.pose.position
  This command displays the robot's estimated position in the odometry coordinate system, which is computed from the robot's sensors (wheel encoders).
- Now, open a second terminal and run the following command:
  $: ros2 topic echo /ground_truth_pose --field pose.position
  This command displays the robot's true position in the world coordinate system, as provided directly by Gazebo (ground truth).

Submit two screenshots showing the outputs of the /ground_truth_pose and /mecanum_drive_controller/odom topics.

c. Start the keyboard control node in a terminal using the following command:
$: ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r cmd_vel:=/cmd_vel
This node publishes velocity commands (/cmd_vel) based on keyboard input.

In addition, open a second terminal and run the following command:
$: ros2 run yahboom_rosmaster_navigation cmd_vel_relay
This relay node converts the velocity commands published by the keyboard controller (/cmd_vel, Twist messages) into the format required by the robot controller (/mecanum_drive_controller/cmd_vel, TwistStamped messages). Without this node running, the robot will not respond to keyboard inputs.

Note: When controlling the robot, make sure the terminal running the keyboard controller is in the foreground; otherwise, the key commands may not be detected.

d. Using the keyboard, manually control the robot so that it moves along a square path in the Gazebo environment.
   - Starting point.
     The robot must start from the initial position (1, 0, 0.05) in the world coordinate system.
   - Square path definition.
     Drive the robot through the following waypoints in order, forming a square:
     - Move from (1, 0, 0.05) to (4, 0, 0.05)
     - Move from (4, 0, 0.05) to (4, 3, 0.05)
     - Move from (4, 3, 0.05) to (1, 3, 0.05)
     - Return from (1, 3, 0.05) back to (1, 0, 0.05)
   - Repetition.
     Repeat this square motion three (3) complete times, always starting and ending at (1, 0, 0.05).
   - Speed control.
     Adjust the robot's linear and angular velocities as needed using the keyboard to follow the square path as accurately as possible.

Data observation and recording
   - While the robot is moving, monitor the robot's position using both coordinate systems:
     - Odometry frame: (/mecanum_drive_controller/odom)
     - World (ground-truth) frame: (/ground_truth_pose)
   - After the robot completes each full square cycle, stop the topic monitoring and record the robot's final position in:
     - the odometry coordinate system, and
     - the world coordinate system
   - **Use the recorded values to complete the table below, entering the final position after each cycle:**

	1st cycle	**2nd cycle**	**3rd cycle**
Odometry	(1.5, 3.1, 0)	(1.5, 3.1, 0)	(1.5, 3.1, 0)
Ground truth	(0.97, 0.15, 0.0)	(1.21, 0.34, 0.0)	(1.34, 0.68, 0.0)

Note: Before filling in the table, make sure the odometry data is converted to the world coordinate system, so that it can be fairly compared with the ground-truth values.

3. Open-loop robot control
   a. Node design

      In this task, you will develop an open-loop motion control ROS node that commands the Rosmasterx3 robot, which uses omnidirectional (mecanum) wheels. The robot must follow the same square path defined in Task 2.d, but this time without manual (keyboard) control.

      Your node must satisfy the following requirements:

      - Path structure (omnidirectional motion only).
        The robot must follow a closed square path composed of:
          - 4 straight-line segments, and
          - direction changes at each corner.

        Since the Rosmasterx3 is an omnidirectional (mecanum) robot, in-place rotations are not allowed in this task.
      - Straight-line timing.
        Compute the time required to traverse each straight segment using:
          - the length of the segment, and
          - the robot's linear velocity magnitude.
      - Direction-change timing (linear only).
        At each corner, the robot must change direction by switching between linear velocities (e.g., forward → strafe left). Keep angular velocity = 0 for the entire motion.
      - Automatic repetition.
        After completing one full square cycle, the robot must automatically repeat the path. The robot should complete three (3) full cycles in total.
      - No position reset.
        Do not reset the robot to its initial position between cycles.
        The objective is to observe how small motion and timing errors accumulate over time in an open-loop control system.

      <u>Submit your ROS node source code, ensuring it is clearly and properly commented.</u>

Note: A reference Python file named *openloop_control.py* is provided in the GitHub repository package at the following path: ~/ros2_ws/src/yahboom_rosmaster/yahboom_rosmaster_projects/src/openloop_control.py.
You may use this file as a starting template for your implementation. Ensure that the square path is executed using only linear velocity commands (no in-place rotations).

b. Data analysis and discussion
Using the data collected during the open-loop control experiment, create the same comparison table as in Task 2.d (keyboard-controlled motion), recording the odometry and ground-truth positions after each cycle.

Low Speed:
(-0.062, 0.14, 0.0)

High Speed:
(-0.12, -0.43, 0.0)

Note: You may repeat the open-loop experiment using at least two different linear speed magnitudes (for example, a lower and a higher speed), while keeping the same square size, same motion sequence, and same number of cycles.

<u>Based on this analysis, answer the following questions:</u>

- Error accumulation.
    - After multiple cycles, does the robot return to its original starting position?
    - Do you observe increasing deviation from the intended square path?
    - How does changing the commanded velocity affect:
        - the final position error after each cycle, and
        - how quickly the error grows over successive cycles?

  Explain your observations based on the characteristics of open-loop control.
- Comparison with keyboard control.
    - Compared to the keyboard-controlled motion in Task 2.d, how does the robot's ability to follow the square path differ?
    - How does velocity choice influence the difference between open-loop control and keyboard control?

  Explain why this difference occurs, considering the absence of feedback, the human corrective action during keyboard control, and the use of omnidirectional motion.

Appendix

Note: In this document, "**$:**" is the symbol of command, but not part of the command. For example, "**$: cd /my_folder**" means you just need to type "**cd /my_folder**" in the terminal.

I. Installing Yahboom Rosmasterx3 simulation package stack

What is Gazebo and why do we need it?

- Gazebo is a 3D physics simulator.
- It lets us test robots without real hardware.
- In ROS Jazzy, Gazebo is not built-in – we must install it and connect it to ROS.

To simulate the RosmasterX3 robot in Gazebo using ROS 2, follow the steps below:

1. Install Gazebo and the ROS-Gazebo bridge
   - Gazebo is the 3D simulator where the robot and world run.
   - ROS-Gazebo bridge packages allow ROS 2 nodes to communicate with Gazebo.
   - Without this bridge, ROS cannot control or receive data from the simulated robot.

   Open a terminal and run.
   Add the Gazebo package repository:
   $: sudo apt update
   $: sudo curl -fsSL https://packages.osrfoundation.org/gazebo.gpg -o /usr/share/keyrings/gazebo-archive-keyring.gpg
   $: echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/gazebo-archive-keyring.gpg] http://packages.osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/gazebo-stable.list

   Install Gazebo and ROS-Gazebo packages:
   $: sudo apt update
   $: sudo apt install -y gz-harmonic
   $: sudo apt install -y ros-jazzy-ros-gz ros-jazzy-ros-gz-sim ros-jazzy-ros-gz-bridge ros-jazzy-ros-gz-image

2. Verify Gazebo is installed correctly:
   $: gz sim --version

```
bebito@LAPTOP-FLFKRVEF:~/ros2_ws$ gz sim --version
Gazebo Sim, version 8.10.0
Copyright (C) 2018 Open Source Robotics Foundation.
Released under the Apache 2.0 License.
```

3. Verify ROS ↔ Gazebo packages exist:
   $: ros2 pkg list | grep ros_gz

```
bebito@LAPTOP-FLFKRVEF:~/ros2_ws$ ros2 pkg list | grep ros_gz
ros_gz
ros_gz_bridge
ros_gz_image
ros_gz_interfaces
ros_gz_sim
ros_gz_sim_demos
```

4. Download the Yahboom Rosmaster simulation code.
   We will download the robot simulation from GitHub.
   - Go into src
     $: cd ~/ros2_ws/src
   - Clone the repository
     $: git clone https://github.com/automaticaddison/yahboom_rosmaster.git
   - After this, you should have: ~/ros2_ws/src/yahboom_rosmaster/

```
bebito@ECE-ES242-QC:~/ros2_ws/src/yahboom_rosmaster$ ls
mecanum_drive_controller  yahboom_rosmaster_bringup      yahboom_rosmaster_gazebo        yahboom_rosmaster_navigation
README.md                 yahboom_rosmaster_description  yahboom_rosmaster_localization  yahboom_rosmaster_system_tests
yahboom_rosmaster         yahboom_rosmaster_docking      yahboom_rosmaster_msgs
```

   This repository contains:
   - Robot description
   - Gazebo simulation
   - Controllers
   - Navigation & Localization configs

5. Install required dependencies automatically: Run rosdep.
   From the workspace root:
   $: cd ~/ros2_ws
   $: rosdep install --from-paths src --ignore-src -r -y
   What this does:
   - Scans all packages
   - Installs everything they depend on
   - Skips packages you already have

6. Build the workspace.
   Now we compile everything:
   $: cd ~/ros2_ws
   $: colcon build --symlink-install

Why --symlink-install?

- It allows you to see changes immediately without needing to rebuild the workspace every time you modify code.
- It's ideal for debugging and testing new code quickly.

During the build, you may see a warning or error related to mecanum_drive_controller:

```
bebito@ECE-E5242-QC:~/ros2_ws$ colcon build --symlink-install
[0.312s] WARNING:colcon.colcon_core.package_selection:Some selected packages are already built in one or more underlay workspaces:
        'mecanum_drive_controller' is in: /opt/ros/jazzy
If a package in a merged underlay workspace is overridden and it installs headers, then all packages in the overlay must sort their include directories by workspace order. Failure to do so may result in build failures or undefined behavior at run time.
If the overridden package is used by another package in any underlay, then the overriding package in the overlay must be API and ABI compatible or undefined behavior at run time may occur.

If you understand the risks and want to override a package anyways, add the following to the command line:
        --allow-overriding mecanum_drive_controller

This may be promoted to an error in a future release of colcon-override-check.
Starting >>> yahboom_rosmaster_msgs
Starting >>> yahboom_rosmaster_bringup
Starting >>> yahboom_rosmaster_description
Starting >>> yahboom_rosmaster_docking
Starting >>> yahboom_rosmaster_gazebo
Starting >>> yahboom_rosmaster_localization
Starting >>> yahboom_rosmaster_navigation
Starting >>> mecanum_drive_controller
Finished <<< yahboom_rosmaster_bringup [1.78s]
Finished <<< yahboom_rosmaster_description [1.88s]
Finished <<< yahboom_rosmaster_localization [2.26s]
Finished <<< yahboom_rosmaster_docking [6.44s]
Finished <<< yahboom_rosmaster_msgs [6.60s]
Starting >>> yahboom_rosmaster_system_tests
Finished <<< yahboom_rosmaster_gazebo [9.55s]
--- stderr: mecanum_drive_controller
In file included from /home/bebito/ros2_ws/src/yahboom_rosmaster/mecanum_drive_controller/include/mecanum_drive_controller/mecanum_drive_controller.hpp:37,
                 from /home/bebito/ros2_ws/src/yahboom_rosmaster/mecanum_drive_controller/src/mecanum_drive_controller.cpp:42:
/opt/ros/jazzy/include/realtime_tools/realtime_tools/realtime_box.hpp:40:151: note: '#pragma message: 'RealtimeBox' is deprecated. Please update your code to use 'realtime_thread_safe_box.hpp' header, and class name 'RealtimeThreadSafeBox' instead.'
   40 |  code to use 'realtime_thread_safe_box.hpp' header, and class name 'RealtimeThreadSafeBox' instead.")  //NOLINT
      |                                                                                                      ^

---
Finished <<< mecanum_drive_controller [11.8s]
Finished <<< yahboom_rosmaster_system_tests [8.27s]
Finished <<< yahboom_rosmaster_navigation [15.9s]
Starting >>> yahboom_rosmaster
Finished <<< yahboom_rosmaster [1.89s]

Summary: 10 packages finished [18.0s]
  1 package had stderr output: mecanum_drive_controller
```

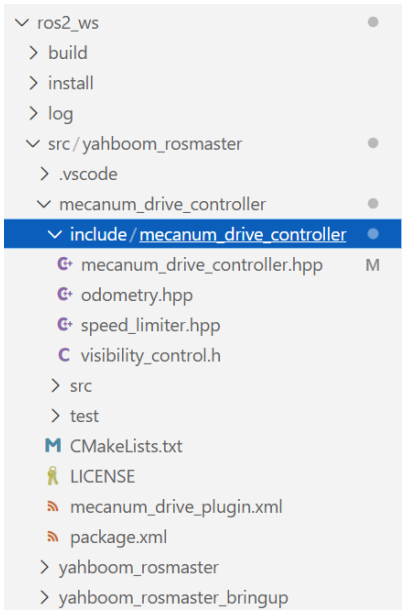
This happens because:

- The repository provides its own mecanum controller
- ROS already has a default one
- We must explicitly allow overriding it

Therefore:

a. Edit the controller file.
   Open the following file:
   ~/ros2_ws/src/yahboom_rosmaster/mecanum_drive_controller/include/mecanum_drive_controller/mecanum_drive_controller.hpp

   Make the following changes:
   - Around line 37.
     Replace: #include "realtime_tools/realtime_box.h"
     With: #include "realtime_tools/realtime_thread_safe_box.h"




```
18  #include <memory>
19  #include <optional>
20  #include <queue>
21  #include <string>
22  #include <vector>
23  #include <array>
24
25  // ROS 2 Controller interface
26  #include "controller_interface/controller_interface.hpp"
27  // Custom headers for mecanum drive functionality
28  #include "mecanum_drive_controller/odometry.hpp"
29  #include "mecanum_drive_controller/speed_limiter.hpp"
30  #include "mecanum_drive_controller/visibility_control.h"
31  // ROS 2 message types
32  #include "geometry_msgs/msg/twist_stamped.hpp"
33  #include "nav_msgs/msg/odometry.hpp"
34  #include "odometry.hpp"
35  // ROS 2 lifecycle and utilities
36  #include "rclcpp_lifecycle/state.hpp"
37  // #include "realtime_tools/realtime_box.hpp"
38  #include "realtime_tools/realtime_thread_safe_box.hpp"
39  #include "realtime_tools/realtime_publisher.hpp"
40  #include "tf2_msgs/msg/tf_message.hpp"
```

- Around line 223.
  Replace: realtime_tools::RealtimeBox<std::shared_ptr<Twist>> received_velocity_msg_ptr_{nullptr};
  With: realtime_tools::RealTimeThreadSafeBox<std::shared_ptr<Twist>> received_velocity_msg_ptr_;

```
222  // Storage for received velocity commands
223  // realtime_tools::RealtimeBox<std::shared_ptr<Twist>> received_velocity_msg_ptr_{nullptr};
224  realtime_tools::RealtimeThreadSafeBox<std::shared_ptr<Twist>> received_velocity_msg_ptr_;
225  std::queue<Twist> previous_commands_;  // Stores last two commands
```

b. Rebuild allowing override:
$: colcon build --symlink-install --allow-overriding mecanum_drive_controller

```
bebito@ECE-ES242-QC:~/ros2_ws$ colcon build --symlink-install --allow-overriding mecanum_drive_controller
Starting >>> yahboom_rosmaster_msgs
Starting >>> yahboom_rosmaster_bringup
Starting >>> yahboom_rosmaster_description
Starting >>> yahboom_rosmaster_docking
Starting >>> yahboom_rosmaster_gazebo
Starting >>> yahboom_rosmaster_localization
Starting >>> yahboom_rosmaster_navigation
Starting >>> mecanum_drive_controller
Finished <<< yahboom_rosmaster_bringup [0.17s]
Finished <<< yahboom_rosmaster_localization [0.19s]
Finished <<< yahboom_rosmaster_description [0.23s]
Finished <<< yahboom_rosmaster_docking [0.24s]
Finished <<< yahboom_rosmaster_gazebo [0.26s]
Finished <<< mecanum_drive_controller [0.27s]
Finished <<< yahboom_rosmaster_navigation [0.32s]
Finished <<< yahboom_rosmaster_msgs [0.40s]
Starting >>> yahboom_rosmaster_system_tests
Finished <<< yahboom_rosmaster_system_tests [0.16s]
Starting >>> yahboom_rosmaster
Finished <<< yahboom_rosmaster [0.37s]

Summary: 10 packages finished [1.14s]
```

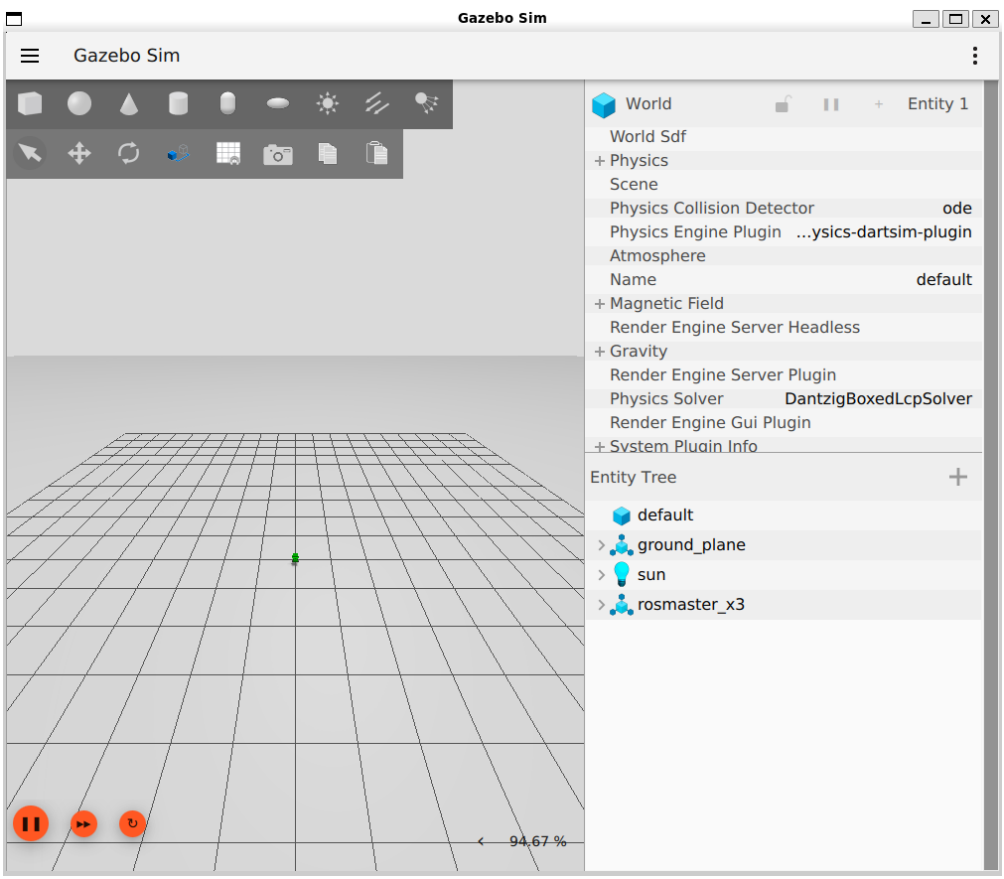
Source the workspace:
$: source ~/ros2_ws/install/setup.bash
Note: Every new terminal must run this command, or ROS will not find the packages.

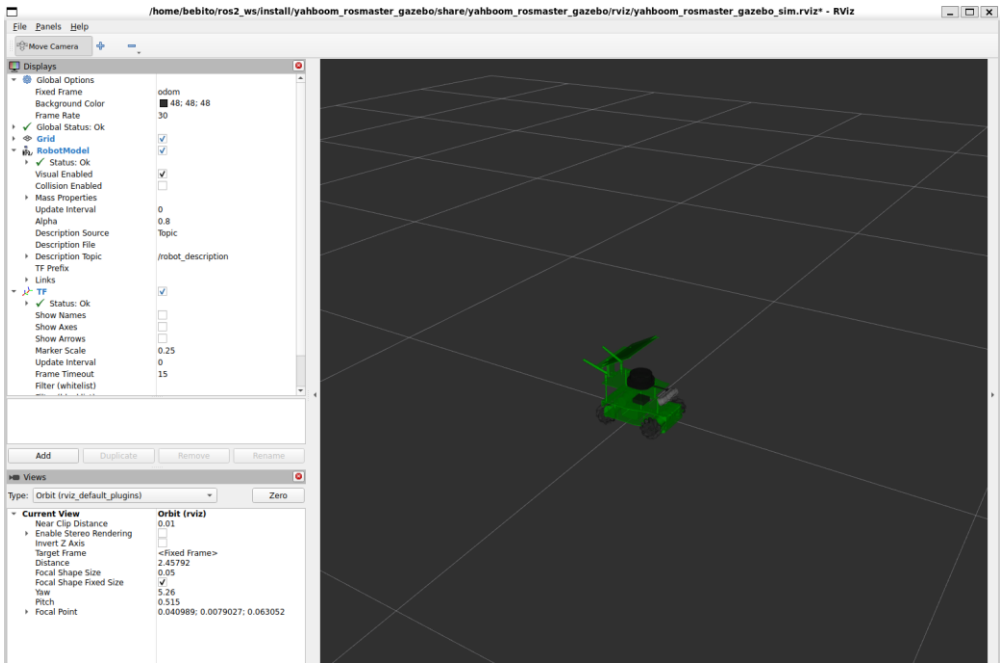
7. Launch the Gazebo simulation.
   - Find available Yahboom packages:
     $: ros2 pkg list | grep yahboom

```
bebito@LAPTOP-FLFKRVEF:~/ros2_ws$ ros2 pkg list | grep yahboom
yahboom_rosmaster
yahboom_rosmaster_bringup
yahboom_rosmaster_description
yahboom_rosmaster_docking
yahboom_rosmaster_gazebo
yahboom_rosmaster_localization
yahboom_rosmaster_msgs
yahboom_rosmaster_navigation
yahboom_rosmaster_system_tests
yahboom_rosmasterx3
```

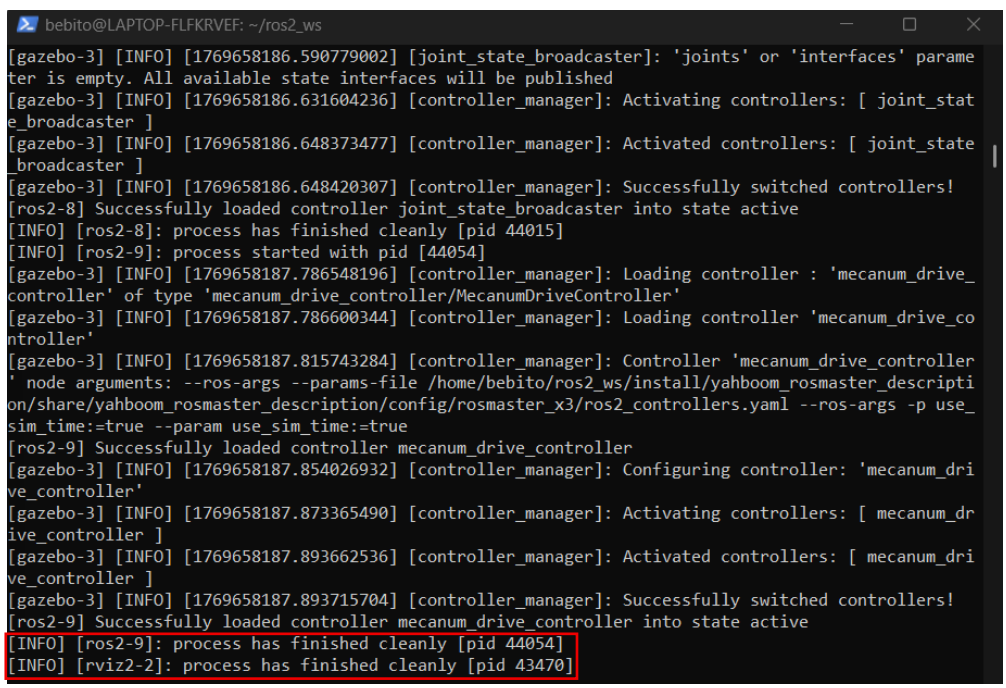
   - Launch Gazebo
     $: ros2 launch yahboom_rosmaster_gazebo yahboom_rosmaster.gazebo.launch.py

What to expect during launch:





Note: The Gazebo simulation may take up to 20 seconds to fully start. When the launch has completed successfully, the terminal will display the message: process has finished cleanly.



```
bebito@LAPTOP-FLFKRVEF: ~/ros2_ws
[gazebo-3] [INFO] [1769658186.590779002] [joint_state_broadcaster]: 'joints' or 'interfaces' parame
ter is empty. All available state interfaces will be published
[gazebo-3] [INFO] [1769658186.631604236] [controller_manager]: Activating controllers: [ joint_stat
e_broadcaster ]
[gazebo-3] [INFO] [1769658186.648373477] [controller_manager]: Activated controllers: [ joint_state
_broadcaster ]
[gazebo-3] [INFO] [1769658186.648420307] [controller_manager]: Successfully switched controllers!
[ros2-8] Successfully loaded controller joint_state_broadcaster into state active
[INFO] [ros2-8]: process has finished cleanly [pid 44015]
[INFO] [ros2-9]: process started with pid [44054]
[gazebo-3] [INFO] [1769658187.786548196] [controller_manager]: Loading controller : 'mecanum_drive_
controller' of type 'mecanum_drive_controller/MecanumDriveController'
[gazebo-3] [INFO] [1769658187.786600344] [controller_manager]: Loading controller 'mecanum_drive_co
ntroller'
[gazebo-3] [INFO] [1769658187.815743284] [controller_manager]: Controller 'mecanum_drive_controller
' node arguments: --ros-args --params-file /home/bebito/ros2_ws/install/yahboom_rosmaster_descripti
on/share/yahboom_rosmaster_description/config/rosmaster_x3/ros2_controllers.yaml --ros-args -p use_
sim_time:=true --param use_sim_time:=true
[ros2-9] Successfully loaded controller mecanum_drive_controller
[gazebo-3] [INFO] [1769658187.854026932] [controller_manager]: Configuring controller: 'mecanum_dri
ve_controller'
[gazebo-3] [INFO] [1769658187.873365490] [controller_manager]: Activating controllers: [ mecanum_dr
ive_controller ]
[gazebo-3] [INFO] [1769658187.893662536] [controller_manager]: Activated controllers: [ mecanum_dri
ve_controller ]
[gazebo-3] [INFO] [1769658187.893715704] [controller_manager]: Successfully switched controllers!
[ros2-9] Successfully loaded controller mecanum_drive_controller into state active
[INFO] [ros2-9]: process has finished cleanly [pid 44054]
[INFO] [rviz2-2]: process has finished cleanly [pid 43470]
```

II. Download the course project repository

Link:
https://github.com/ClauORP/yahboom_rosmaster_projects.git
This repository will be updated regularly throughout the semester as projects progress.
You will use the files inside this repository to develop your projects.

- Contains starter code, configs and templates.
- Updated regularly with changes and additions for upcoming projects.
- New folders/files will appear over time.

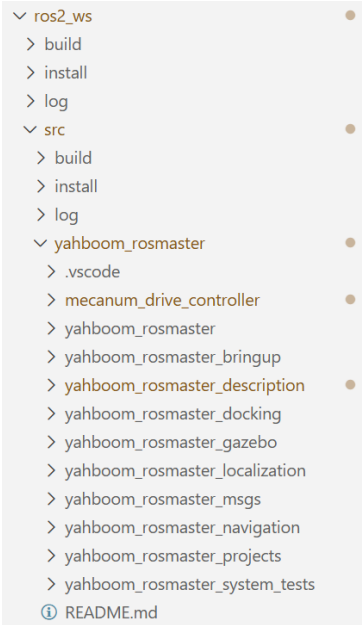
Note: Cloning the repository once is not enough - you will need to pull updates.

1. Clone the repository.
   Open terminal and run:
   $: cd ~/ros2_ws/src/yahboom_rosmaster/
   $: git clone https://github.com/ClauORP/yahboom_rosmaster_projects.git

2. Understand the "update" workflow.
   Because this repository will change over time, you need to update it regularly.
   Whenever the instructor announces a new update, you need to run:
   $: cd ~/ros2_ws/src/yahboom_rosmaster/yahboom_rosmaster_projects
   $: git pull
   This downloads the latest changes.

3. Rebuild after updates.
   Some updates may involve new ROS code that needs to be compiled. If that happens, you need to rebuild the workspace after pulling updates.
   $: cd ~/ros2_ws
   $: colcon build --symlink-install
   $: source ~/ros2_ws/install/setup.bash

4. Verify the workspace setup.
   Before starting the project, make sure everything is set up correctly.
   You should see the following workspace structure:



5. Start the Gazebo simulation environment.
   Now, you can start the Gazebo simulation environment for this project.
   $: ros2 launch yahboom_rosmaster_projects gz_home_like.launch.py
   Note: For this project, you will always use this environment.

III. Understanding coordinate systems

When working with robots in Gazebo and ROS 2, understanding different coordinate systems (also called frames) is essential for correct navigation, localization, and control. A coordinate system defines where the robot is measured from and how its position is described.

1. **World coordinate system (Global frame)**
   "You spawned at this exact position in the simulation."
   The world coordinate system in Gazebo is the fixed, global reference frame of the simulation environment. This coordinate system represents the "absolute" frame of the simulated world and does not move or change during the simulation, like a map in the real world.
   In the Gazebo visualization window, the world coordinate system's origin is fixed at the center of the environment.

   Note: On the floor of the Gazebo environment, there is a 20×20 grid, where each square represents 1 meter $\times$ 1 meter. After launching the Gazebo world, the robot's default initial position can be observed directly in this global frame. For example, the robot starts at coordinates (1, 0, 0.05) in the world coordinate system.
   This frame is often referred to as <u>ground truth</u>, since it represents the robot's exact position in the simulation.

2. **Odometry coordinate system (Odom frame)**
   "I start counting from zero and track how you move from there."
   The odometry coordinate system is a local reference frame that is attached to the robot. Unlike the world frame, the odom frame starts at the robot's initial position when the simulation begins and moves relative to that starting point.
   This coordinate system is not fixed in Gazebo. Instead, the robot estimates its position in the odom frame using its onboard sensors, such as wheel encoders and the IMU. Because this estimation is based on sensor measurements, small errors can accumulate over time, causing the odometry position to drift.

   Note: The robot's initial position in the odometry coordinate system is always defined as (0, 0, 0), even if the robot starts at a different location in the world coordinate system.